00_PROJECT_MEMORY.md

PROJECT MEMORY

Version: 1.0

Status

Living Document

This document records the current state of the project.

Every engineer must read this document before beginning work.

Every completed task must update this document.

This document is the single source of truth for project progress.

Never overwrite previous decisions without documenting why.

Project Information

Project

Secure Dance Academy Management System

Architecture

Feature-Based Clean Architecture

Framework

Next.js 15

Language

TypeScript

Styling

Tailwind CSS

shadcn/ui

Database

Supabase PostgreSQL

ORM

Prisma

Authentication

Supabase Authentication

Deployment

Vercel

Docker

Testing

Jest

Playwright

OWASP ZAP

SonarQube

Status

Planning Phase

Project Goals

Build a production-quality secure web application.

Follow Secure by Design principles.

Demonstrate professional software engineering.

Satisfy all coursework requirements.

Create a portfolio-quality application.

Current Progress

Project Planning

Status

Completed

Requirements Engineering

Status

Not Started

Architecture Design

Status

Not Started

Database Design

Status

Not Started

UI Design

Status

Not Started

Backend Development

Status

Not Started

Frontend Development

Status

Not Started

Security Implementation

Status

Not Started

Testing

Status

Not Started

Formal Methods

Status

Not Started

Documentation

Status

Not Started

Deployment

Status

Not Started

Final Review

Status

Not Started

Completed Documents

Completed

Project Director Handbook

Requirements Engineer Handbook

Security Architect Handbook

Solution Architect Handbook

Database Architect Handbook

UI UX Lead Handbook

Backend Lead Handbook

Frontend Lead Handbook

QA Security Lead Handbook

Formal Methods Engineer Handbook

Technical Writer Handbook

Principal Engineer Handbook

Project Context

Engineering Rules

Project Memory

These documents define how the engineering team operates.

They should rarely change.

Approved Technologies

Frontend

Next.js 15

TypeScript

Tailwind CSS

shadcn/ui

Backend

Next.js Route Handlers

Server Actions

Prisma ORM

Database

Supabase PostgreSQL

Authentication

Supabase Authentication

Validation

Zod

Forms

React Hook Form

Icons

Lucide React

Tables

TanStack Table

Charts

Recharts

Testing

Jest

Playwright

Postman

Security

OWASP ZAP

SonarQube

Deployment

Docker

Vercel

GitHub

No alternative technologies should be introduced without approval.

Approved Architecture

Feature-Based Architecture

Clean Architecture

REST API

Server Components First

Client Components Only When Needed

Repository Pattern

Service Layer

DTO Pattern

RBAC

Audit Logging

Feature Isolation

These architectural decisions are fixed.

Approved Coding Standards

TypeScript Only

Strict Type Safety

Reusable Components

Small Functions

Single Responsibility

Descriptive Naming

Server Validation

Consistent Error Handling

Professional Documentation

Do not violate these standards.

Security Decisions

Authentication

Supabase Authentication

Authorization

Role-Based Access Control

Session Management

Secure Cookies

Validation

Zod

Password Security

Supabase Authentication

Audit Logging

Enabled

Environment Variables

Required

OWASP Compliance

Required

NIST Principles

Required

These decisions should remain consistent throughout development.

Project Modules

Authentication

Users

Roles

Artists

Parents

Coaches

Attendance

Performances

Injuries

Medical Records

Activities

Reports

Dashboard

Notifications

Audit Logs

Settings

Every future feature should belong to one of these modules.

Outstanding Decisions

Multi-language Support

Pending

Dark Mode

Pending

Email Provider

Pending

Notification Strategy

Pending

Backup Automation

Pending

Analytics Dashboard

Pending

Record decisions here as they are made.

Known Constraints

Coursework Deadline

Must satisfy assessment requirements.

Professional code quality.

Strong security.

Formal methods required.

Comprehensive documentation required.

Portfolio quality expected.

Never compromise these constraints.

Technical Debt Register

No technical debt recorded.

Whenever technical debt is introduced:

Record

Reason

Impact

Priority

Owner

Resolution Plan

Never hide technical debt.

Risk Register

Current Risks

No major risks identified.

Update whenever:

Architecture changes.

Security risks appear.

Dependencies change.

Performance issues arise.

Infrastructure changes.

Bug Register

No bugs recorded.

When defects are found record:

ID

Title

Severity

Module

Status

Owner

Resolution

Verification

This register should always remain current.

Architecture Decision Record

Record every important decision.

Decision Number

Date

Decision

Reason

Alternatives Considered

Impact

Owner

Do not change architecture silently.

Feature Progress

Every feature should follow this workflow.

Planned

Approved

In Development

Testing

Review

Completed

Rejected

Cancelled

Update feature status continuously.

Deliverable Progress

Requirements

Pending

Architecture

Pending

Database

Pending

Frontend

Pending

Backend

Pending

Security

Pending

Testing

Pending

Formal Methods

Pending

Documentation

Pending

Deployment

Pending

Presentation

Pending

Final Report

Pending

Use this section to monitor overall progress.

Lessons Learned

Record important discoveries.

Examples

Architecture improvements.

Better security approaches.

Performance optimizations.

Testing improvements.

Documentation improvements.

Prevent repeating mistakes.

Future Improvements

Record ideas that should not be implemented immediately.

Examples

Mobile Application

Push Notifications

Payment Integration

Calendar Synchronization

Multi-Tenant Support

Artificial Intelligence Assistant

Keep future ideas separate from current scope.

Next Task

Current Phase

Project Planning Complete

Next Phase

Requirements Engineering

Primary Owner

Requirements Engineer

Expected Deliverables

Software Requirements Specification

Use Cases

User Stories

Business Rules

Security Requirements

Risk Analysis

STRIDE Model

Acceptance Criteria

Requirements Traceability Matrix

Only begin architecture after requirements are approved.

Memory Update Rules

Every completed task must update this document.

Never delete historical decisions.

Never overwrite architecture without justification.

Never remove completed milestones.

Always record:

Major decisions

Architecture changes

Technology changes

Security changes

Completed features

Known issues

Technical debt

Project risks

This document must always represent the current state of the project.

Project State

Overall Status

Planning Complete

Engineering Team

Ready

Architecture

Pending

Development

Not Started

Testing

Not Started

Documentation

Not Started

Final Review

Pending

Project Health

Excellent

Confidence Level

High

This document should evolve throughout the project and become the first file every engineer reads before starting work. It preserves context, maintains consistency, and ensures the project grows in a controlled and professional manner.